COM115 Week 1 Lab

Introduction to Python and Jupyter Notebooks

You have probably started by opening the pdf version of this file. If so, you should follow the instructions below to open the *.ipynb version of this file in the Jupyter Notebook App.

The Jupyter Notebook App is a tool for creating documents (notebooks) containing both live code and text, as well as visualizations, etc. Various programming languages can be used, but in COM115 we will be using only Python.

Working from the labs

Move the *.ipynb version of this file to your P: drive

In the labs, files created on the C: drive will be lost overnight.
For your files to save, you must create them in your P: drive. To do this, follow steps 1-3 below.

1. Create a COM115 folder on your P: drive.
2. In your COM115 folder, create a Week 1 subfolder.
3. Download the file Week1_lab.ipynb from Blackboard, and move it to your Week 1 subfolder.

Open the *.ipynb version of this file in the Jupyter Notebook App

Now we need to open the Jupyter Notebook App and use it to open the file Week1_lab.ipynb.
Navigating to the P: drive on Jupyter notebook requires starting it from the command prompt. Please follow steps 4-6 below.

4. From the Windows menu, select Anaconda3, and run the Anaconda prompt.
5. From the prompt, enter the command: jupyter notebook --notebook-dir=P: If a dialog opens asking you what program you wish to use, select a browser, preferably chrome.
6. In the browser window that opens, navigate to your file Week1_lab.ipynb and click on it to open.

Solutions to the lab exercises will be released ahead of next week's class.
You should try your best to get the exercises completed each week, before the solutions are released.

Working from a home computer

If you wish to install the Jupyter notebook app on your own computer / laptop at home, you should install Anaconda from:

https://www.anaconda.com/products/individual (https://www.anaconda.com/products/individual)

Create a copy of the original notebook to work on

One thing you should do at the outset is to save this noteback as a new notebook, so that you can change it as much as you want, but still go back to the original. From the menu, go to *File -> Make a Copy...*. This creates a copy of your notebook called *Week1_Lab-Copy1* in your current folder. You can

change the name of the notebook (and the file) by going to *File -> Rename* and changing it to *Week1_Lab_my_version* for example, or by editing the name of the notebook at the top of the page beside the Jupyter symbol (just above the menu).

Remainder of this notebook

The rest of this notebook is intended as an introduction to key features of Python and Jupyter Notebooks, irrespective of whether you have used Python before or not. It is not intended to be exhaustive, but will cover material that will be relevant in this module. In addition to this material, you are strongly encouarged to develop your knowledge of Python further through the use of other resources such as

- The Python tutorial - https://docs.python.org/3/tutorial/ (https://docs.python.org/3/tutorial/)

Both Python and Jupyter Notebooks are used in various areas, including data science, so it would be well worth your while to develop your skills in this area as much as possible.

The Jupyter Notebook Interface

Cell Types

We need to distinguish between different types of cells. This cell is a **Markdown cell** whereas the next cell below is a **Code cell**. Markdown cells are for formatting text rather than for running code. To see how to format headings, use italics and bold font, for example, you can go into edit mode by double-clicking on a cell. Try it for this cell. To execute the cell (and so produce the formatted text), you can go to *Cell / Run Cells* or use the shortcut *Ctrl-Enter*. (You can find other keyboard shortcuts under the *Help* menu.) Markdown is very useful for mathematical notation such as $\sqrt{2}$. For further details on Markdown see Markdown in Jupyter Notebook (https://www.datacamp.com/community/tutorials/markdown-in-jupyter-notebook).

The next cell below is a Code cell.

Type `print('Hello World!')` into the cell and run the cell by either:

1. clicking the run button to the immediate left of the cell itself
2. clicking the run button on the toolbar at the top of the screen
3. clicking in the cell, and then pressing Shift+Enter

If it ran ok, the output will be displayed below the cell. Try running subsequent code cells as you work your way through this notebook.

```
In [ ]:
```

Simple Calculations and Formatting

Below is a cell showing a calculation of $\frac{25 - 4.5^2}{3}$

In [ ]:

```
# Using Python as a calculator
(25 - 4.5**2) / 3   # Notice how to square a number
```

We could also use the `print` function:

In [ ]:

```
print((25 - 4.5**2) / 3)
```

Or using a variable:

In [ ]:

```
x = (25 - 4.5**2) / 3
print(x)
```

We can also format the output. One way to do it in Python is to use the `format` function as follows `print('{:X.Yf}'.format(1.234567))` where X and Y need to be replaced by suitable integers. This is illustrated in the cell below:

(For details on formatting, see https://www.python-course.eu/python3_formatted_output.php (https://www.python-course.eu/python3_formatted_output.php) and https://docs.python.org/3/tutorial/inputoutput.html (https://docs.python.org/3/tutorial/inputoutput.html))

In [ ]:

```
x = (25 - 4.5**2) / 3
print('{:5.3f}'.format(x))
```

You should try out a few other calculations like the one above. To do this, you have a few options. You can edit a current cell, such as the one above, to do a different calculation, but it is better to insert a new cell. You can do this by a) clicking on the cell above and then clicking *Insert -> Insert Cell Below* from the menu or b) when executing a cell you can click *Cell -> Run Cells and Insert Cell Below* from the menu (or alternatively use the shortcut *Alt-Enter*).

Here are a couple of things you should check out. Confirm that unlike some other languages Python does not use integer division for `7/3`. For integer division you would instead use `7//3`.

Exercise 1

You should also make sure you understand precedence in Python (https://docs.python.org/3/reference/expressions.html?highlight=precedence). For example

- Is 4 / 3 * 2 the same as (4 / 3) * 2 or 4 / (3 * 2)?
- What is the result of 5 + 8 / 4 - 3 * 6?

You should try these out in the Code cell below, using the `print` function to display output.

In [ ]:

```

```

Including Code

Basic programming structures

Of course, as a programming language, Python provides the usual sorts of branching (`if` , `if else`) and repetition (`for` , `while`) structures. Try out the code below.

Note 1 In Python you do not need to declare the type of a variable.

Note 2 For programming structures such as `if` and `while` , the relevant code block that gets executed is defined by indentation, i.e. all the indented code is executed if the condition is true. The first unindented line of code is executed after the `if` or `while` statement is completed. This is very different from languages such as C and Java where { } are used.

In [ ]:

```
# Determine the number of items in different price ranges

new_item = 'y'
num_under_10 = 0
num_10to20 = 0
num_over_20 = 0

while new_item == 'y' or new_item == 'Y':
    price = float(input("Please enter the price of the item: "))

    if price < 10:
        num_under_10 = num_under_10 + 1
    elif price < 20:
        num_10to20 = num_10to20 + 1
    else:
        num_over_20 = num_over_20 + 1

    new_item = input('Do you have another item? ' +
                     '(Enter y for yes): ')

print('\nNumber of items under £10:', num_under_10)
print('Number of items between £10 and £20:', num_10to20)
print('Number of items that are £20 or more:', num_over_20)
```

Exercise 2

Copy the above code to the cell below and then modify it to determine the total and average of the items purchased. You should exclude any items that have a negative value from your calculation.

In [ ]:

Note about Jupyter Notebooks

You will have noticed that when you run a cell, a number appears beside it at the left hand side. These show us the order in which the cells have been executed. Unlike a typical program, the code does not have to be executed in a particular order. Consider the following three cells. If you execute them in

order, the value of x displayed by the second cell is the value specified in the first of the three cells, i.e. 5, as you would expect. But suppose you execute the second of these cells after you execute the third one, you will find that the value of x displayed is 10.

To make your notebook readable, you should avoid writing code that depends on cells that appear later, but if something goes wrong in a calculation, you should check that it isn't related to this issue.

In [ ]:
```
x = 5
```

In [ ]:
```
print('x is',x)
```

In [ ]:
```
x = 10
```

Functions

The following cell includes a function to calculate the mean of a number of randomly generated integers. The function is then called with appropriate arguments. Try to determine exactly how the code works and what the parameters represent. How would you modify the code to print out each random number generated? Or count up the number of even numbers generated?

In [ ]:
```
# Import Python's random module
import random

def meanRandom(n,a,b):
    sum = 0

    for i in range(1,n+1): # i will take on the values 1 up to n
        sum = sum + random.randint(a,b)

    return sum/n

# Call the function meanRandom
x = meanRandom(100,1,6)
print('x is', x)
```

Functions can also return a boolean value of True or False. What does the following code do?

In [ ]:
```
def func(n,a):
    if n > 2*a:
        return True
    else:
        return False

# Call the function
x = func(50,25)
print(x)
```

Exercise 3

Write a function to determine whether a given number *n* is divisible by either of two other numbers, *a* or *b*.

Hint: Your function should have three parameters for *n*, *a* and *b* like the `meanRandom` function above. To decide whether one number is divisible by another you can use the modulus operator `%` to determine the remainder after division, i.e. `n % a` gives the remainder of *n* after division by *a*.

Then use a `for` loop to call your function for each number up to 100, printing out all the numbers that are divisible by 3 or 5.

Bonus question: Write a function to determine whether a given number *n* is divisible by either of two

In [ ]:

```

```

In [ ]:

```

```

Arrays in Numpy

[Numpy (https://numpy.org/devdocs/user/quickstart.html)](https://numpy.org/devdocs/user/quickstart.html) is a Python library that is widely used for scientific computing and data science. So although *Lists* can be used in Python for handling arrays, Numpy provides us with more scope, especially for dealing with matrices, which are two-dimensional arrays and we will look at them later in the module. One dimensional arrays are called vectors.

Here's a simple example of how we can create, print and access the elements of an array.

In [ ]:

```
# Import Numpy
import numpy as np

# Create an array a
a = np.array([3, 6, 5, 4, 2])

# Print the array
print(a)

# Print the first and last elements of the array
# Note: the index of the first element is 0 and the index of the last element
# is one less than the number of elements
print(a[0])
print(a[4])
```

Creating arrays

Numpy provides different ways to create arrays, including the following.

(In the cells below there is no import Numpy statement, so it is assummed you have already run the previous cell above first.)

In [ ]:

```
# To create an array of numbers 0, 1, ..., 9
b = np.arange(10)
print('b =',b)

# Or from 3 to 12
c = np.arange(3,13)
print('c =',c)

# To create an array with 21 equally space numbers between 0 and 10
d = np.linspace(0,10,21)
print('d =',d)

# To create an array of zeros
e = np.zeros(10)
print('e =',e)

# Or ones
f = np.ones(10)
print('f =',f)

# Or the standard way of creating an array with specified numbers
g = np.array([7, 6, 22, 13, 4])
print('g =',g)
```

Mathematical operations on arrays

In [ ]:

```
# We can create a copy of the array
# Note: If instead we use b = a, that works differently since b and a would refe
# array so changing an element of b would also change the same element of a
b = np.copy(a)
print('b =',b)

# We can change the value of particular elements, e.g. to square an element
b[2] = b[2]**2
print('b =',b)

# We can also apply mathematical operations to all elements of the array, e.g. a
b = a + 2
print('b =',b)

# Or multiply the square root of each number by 2
b = 2 * np.sqrt(a)
print('b =',b)
```

For loops and arrays

In [ ]:

```
# We can access the array element by element using a for loop
for i in a:
    if i % 2 == 0:
        print(i, 'is even')
    else:
        print(i, 'is odd')

# Note: in the above for loop i refers to the values each element of the array a
# So i refers to the values 3, 6, 5, 4, 2
# Here's a version where i refers to each index in the array in turn
# range(5) takes on the values 0, 1, 2, 3, 4
for i in range(5):
    if a[i] % 2 == 0:
        print(a[i], 'is even')
    else:
        print(a[i], 'is odd')
```

Exercise 4

Suppose a group of students (listed in the same order in each case) obtained the following marks out of 100 for the coursework and exam in a module.

- Coursework: 54, 67, 40, 82, 72, 43, 33, 73, 53, 80, 42, 63, 88, 29, 55, 60.
- Exam: 59, 61, 37, 78, 67, 36, 40, 68, 48, 59, 36, 60, 91, 25, 54, 64.

Write code to represent this information in two arrays and then determine the overall mark for each student assuming an equal weighting between coursework and exam. Marks should be rounded up to the nearest integer. Look at functions such as `round`, `floor`, `ceil` and `rint` from Numpy and choose the most appropriate one. See https://docs.scipy.org/doc/numpy/reference/ (https://docs.scipy.org/doc/numpy/reference/)

Use a for loop to display the overall marks for each student with the letter 'D' after all the students who achieved a distinction (70% or higher), 'F' for all those who failed (less than 40%) and 'P' for all the other students who passed.

In [ ]: